

Voice command shopping assistant — architecture & design

1. Tech stack

Layer	Technology	Why
Frontend	React + Vite	Fast to stand up, natural fit for a voice/mobile UI
Voice capture	Web Speech API (browser native)	Zero backend audio infra, built-in multilingual support via <code>lang</code>
Backend	Spring Boot 3.x + Java	Familiar, production-shaped REST layer, fast to build in
NLP / intent extraction	Spring AI <code>ChatClient</code> + structured output	Maps LLM response straight into a typed Java DTO, no manual JSON parsing
Database	PostgreSQL (single instance)	Everything — catalog, lists, history — fits relationally
Fuzzy search	<code>pg_trgm</code> extension	Typo-tolerant product matching without a vector store
Recommendations	Deterministic rule/co-occurrence engine	Explainable, testable, no LLM dependency for the core feature
Deployment	Render/Railway (backend), Vercel/Netlify (frontend), Neon/Supabase (Postgres)	Free tiers, minimal setup

Explicitly not used: Neo4j, ChromaDB/pgvector, LangChain, LangGraph, microservices, Kafka, Redis. None of these are justified by the assignment's actual requirements, and each would burn hours of an 8-hour budget on infrastructure instead of product. If asked in review why they're absent: *"the problem doesn't require graph traversal, semantic retrieval, or multi-step agent orchestration — a modular monolith with one database covers it, and staying deterministic where possible keeps the system explainable and testable."*

2. Core design principle

The LLM interprets. It never acts.

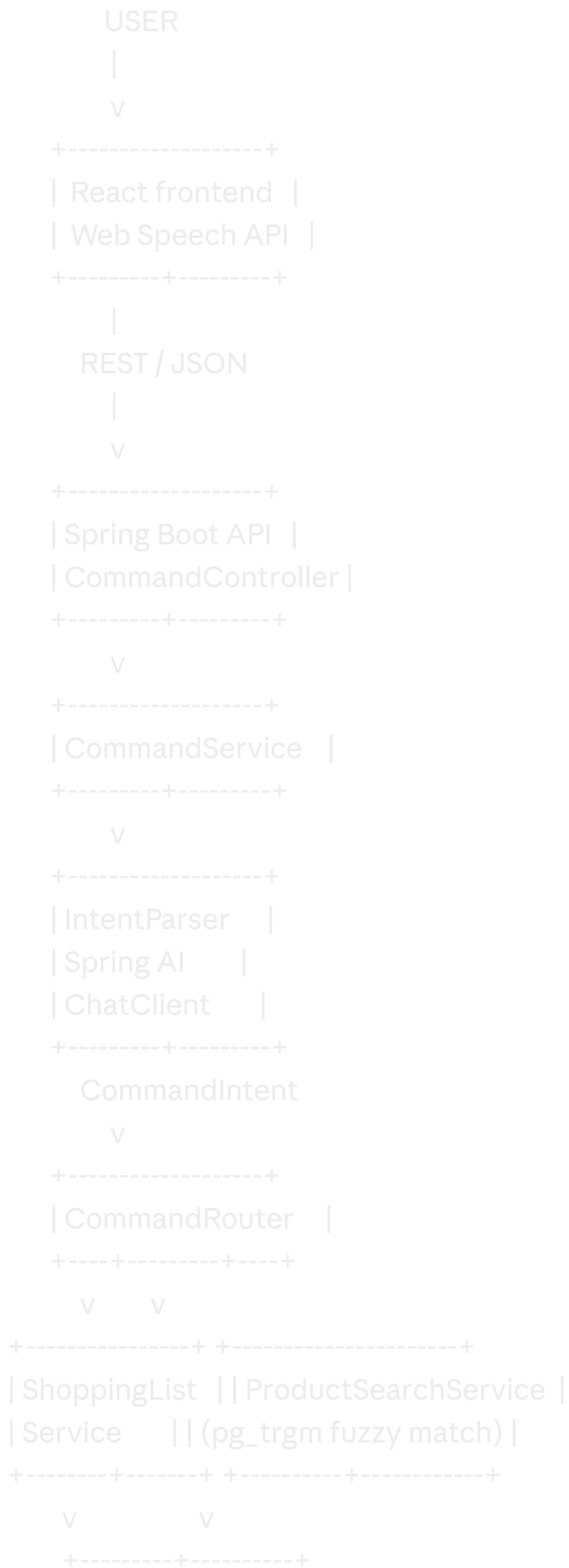


```
voice → transcript → LLM → CommandIntent (typed DTO) → validation → domain service → database
```

The model's only job is turning natural language into a structured, validated command object. Every actual mutation (adding an item, changing quantity, running a query) happens in ordinary, deterministic Java code that you can unit test without touching an API. This is the single most important thing to get right — and the easiest thing to explain well in review.

3. High-level architecture





```

      v
+-----+
| PostgreSQL |
| single database |
+-----+

```

Separately (not on the voice path):

PurchaseHistory --> RecommendationEngine --> ranked suggestions --> React UI

Five logical components:

Presentation — VoiceInput, ShoppingList, SuggestionPanel, ProductSearch, LanguageSelector

API — CommandController, ShoppingListController, ProductController, SuggestionController

Application — CommandService, ShoppingListService, ProductSearchService, RecommendationService

Intelligence — IntentParser (Spring AI), RecommendationEngine (rule-based)

Persistence — ProductRepository, ShoppingListRepository, PurchaseHistoryRepository

4. Low-level design

Package structure



```

backend/src/main/java/com/you/shoppingassistant/
├── controller/ CommandController, ShoppingListController, ProductController,
SuggestionController
├── service/ CommandService, ShoppingListService, ProductSearchService,
RecommendationService
├── ai/ IntentParser, CommandIntent
├── recommendation/ RecommendationEngine, PersonalHistoryStrategy,
CoOccurrenceStrategy
├── repository/ ProductRepository, ShoppingListRepository, PurchaseHistoryRepository
├── entity/ Product, ShoppingListItem, PurchaseHistory, User
└── dto/ CommandRequest, CommandResponse, SuggestionResponse

```

Database schema



```
products
  id, name, category, brand, price, unit, in_stock, season, substitute_group

shopping_list_items
  id, user_id, product_id, quantity, unit, status

purchase_history
  id, user_id, product_id, quantity, purchased_at

co_occurrence_seed
  item_a, item_b, weight  -- small hand-seeded table, not mined
```

CommandIntent DTO

```
□

CommandIntent
  intent    ADD_ITEM | REMOVE_ITEM | UPDATE_QUANTITY | SEARCH_PRODUCT |
SHOW_LIST | UNKNOWN
  productName
  brand
  quantity
  unit
  maxPrice
  minPrice
```

API surface

```
□

POST /api/commands      { text }      -> { intent, message, updatedList }
GET  /api/shopping-list                -> current list
POST /api/products/search { query, maxPrice } -> matched products (pg_trgm)
GET  /api/suggestions                -> ranked recommendations
```

Everything voice-driven goes through the single `/api/commands` endpoint — “remove milk” resolves to `REMOVE_ITEM` inside the same intent schema, so there’s no need for a separate voice-remove route.

Command flow (add 2 bottles of water)

```
□
```

```

"Add 2 bottles of water"
|
Web Speech API transcribes
|
POST /api/commands { text: "Add 2 bottles of water" }
|
Spring AI ChatClient -> CommandIntent
{ intent: ADD_ITEM, productName: "water", quantity: 2, unit: "bottle" }
|
CommandRouter -> ShoppingListService.addItem()
|
PostgreSQL insert
|
CommandResponse { message: "Added 2 bottles of water", updatedList: [...] }
|
React re-renders list + shows confirmation

```

5. Recommendation engine (scoped for 8 hours)

Two strategies, not four — enough to demonstrate the pattern without burning the clock on data you don't have yet:

```

□
RecommendationEngine
├── PersonalHistoryStrategy -- days since last purchase vs. average interval
├── CoOccurrenceStrategy   -- looks up co_occurrence_seed, ~15 hand-picked pairs
└──

```

$$\text{score} = 0.6 \times \text{personalHistoryScore} + 0.4 \times \text{coOccurrenceScore}$$

Note in the README that these weights are illustrative, chosen for the demo, not empirically tuned.

`seasonal` and `substitutes` are real requirements in the brief — if time remains, add them as two more classes implementing the same `RecommendationStrategy` interface (season match against a `season` column; substitute lookup by `substitute_group`). Both are cheap once the interface exists. If time runs out, say so plainly in the write-up rather than leaving a half-wired feature: *"seasonal and substitute recommendations are designed for via the same strategy interface but not implemented in this pass."*

Why no association-rule mining: proper support/confidence/lift mining needs a transaction dataset large enough to produce real signal — building that synthetic dataset alone can eat an

hour you don't have. A hand-seeded co-occurrence table gets the same demoable behavior for a fraction of the cost, and you can say exactly that if asked.

6. Deployment

Frontend → Vercel or Netlify (static build)

Backend → Render or Railway (Spring Boot free tier)

Database → Neon or Supabase (managed Postgres, `pg_trgm` available as an extension)

LLM provider → whichever free tier Spring AI supports that you're comfortable with (OpenAI, Gemini, or a local Ollama model if you want zero API cost)

7. Build plan (8-hour budget)

Time	Task
0:00–1:00	Mic capture + transcript displayed in React
1:00–3:00	<code>/api/commands</code> end to end: Spring AI intent extraction → CommandRouter → add/remove/update working
3:00–4:30	List CRUD + product search with <code>pg_trgm</code> + minimal UI
4:30–6:00	Recommendation engine: two strategies + <code>/api/suggestions</code>
6:00–7:00	Deploy frontend + backend + DB
7:00–8:00	README, 200-word write-up, buffer for whatever broke

Multilingual support and price-range filtering are both cheap add-ons once the core intent schema exists (a `lang` toggle and one more DTO field respectively) — good use of any time left over, not something to architect for up front.